

NEXORADB — LANGUAGE REFERENCE V1.0

NexoraQL

NexoraDB — Document Store · Live Graph · Static Graph · مرجع کامل زبان کوئری · Algorithms

VERSION

MVP 1.0.0

GRAPH

Live + Static

STORAGE

RocksDB

BASE / پایه

SQL-like

فهرست مطالب / TABLE OF CONTENTS

Data Types	02\$ — انواع داده	Concepts	01\$ — مفاهیم پایه
DDL — Index & Foreign Key	04\$ — ایندکس و Foreign Key	DDL — Collections	03\$ — مدیریت Collection
DML — SELECT	06\$ — جستجو و خواندن	DML — INSERT	05\$ — درج داده
DML — DELETE	08\$ — حذف	DML — UPDATE	07\$ — به روزرسانی
TCL — Transactions	10\$ — تراکنش‌ها	DQL — JOIN	09\$ — Join و Lookup
GDL — Node/Edge Mapping	12\$ — ساخت گراف از اسناد	GDL — Graph Define	11\$ — تعریف گراف
GQL — Traversal	14\$ — Traversal گراف	GML — Build و Render	13\$ — Build/Render گراف
Admin — System	16\$ — مدیریت سیستم	GAL — Algorithms	15\$ — الگوریتم‌های گراف

NexoraQL چیست؟ یک زبان کوئری با سینتکس نزدیک به SQL است که برای NexoraDB طراحی شده. از آن می‌توان برای عملیات Document Store (مانند MongoDB)، ساخت و کوئری گراف (مانند Neo4j) یا ترکیب هر دو استفاده کرد. هر کوئری از طریق Parser Python به توابع ++C هدایت می‌شود.

● قوانین سینتکس / Syntax Rules

حساسیت به حروف	کلمات کلیدی UPPER — شناسه‌ها lower
پایان دستور	نقطه‌ویرگول ; اجباری
رشته‌ها	تک کوتیشن '...' یا دو کوتیشن "..."
کامنت تک خط	-- این کامنت است
کامنت چندخط	/* ... */
فیلدهای تودرتو	نقطه address.city
آرایه‌ها	کروشه [3, 2, 1]
JSON inline	آکولاد {"key": "val"}

دسته بندی دستورات / Command Categories ●

ساخت ساختار — Data Definition	DDL
Data Manipulation — CRUD	DML
جستجوی پیشرفته — Data Query	DQL
Transaction Control	TCL
تعریف گراف — Graph Definition	GDL
Graph Management — Build/Render	GML
Graph Query — Traversal	GQL
اجرای الگوریتم — Graph Algorithm	GAL
مدیریت — System Admin	SYS

● انواع اسکالر / Scalar

نوع	مثال
STRING	'ali'
INT32	42
INT64	999999999
FLOAT64	3.14
BOOL	true / false
NULL	null

● انواع مرکب / Compound

نوع	مثال
ARRAY	['a', 'b', 'c']
OBJECT	{x": 1"}
BINARY	داده خام

● عملگرهای مقایسه / Operators

برابر	EQ یا =
نابرابر	NEQ یا !=
بزرگ‌تر	=< <
کوچک‌تر	=> >
عضو لیست	IN
غیرعضو	NOT IN
فیلد وجود دارد	EXISTS
الگوی regex	LIKE
شروع با	STARTS WITH
حاوی	CONTAINS

DocEngine::CreateCollection

DDL — CREATE COLLECTION

```
-- Schema بدون ساده‌ترین شکل
CREATE COLLECTION users;

-- Schema کامل
CREATE COLLECTION users (
  _id      STRING   REQUIRED UNIQUE,
  username STRING   REQUIRED UNIQUE,
  email    STRING   REQUIRED UNIQUE,
  age      INT32,
  bio      STRING,
  active   BOOL     DEFAULT true,
  created_at INT64   REQUIRED,
  tags     ARRAY,
  address  OBJECT
);

-- سختگیر (فیلد ناشناخته رد می‌شود) Schema
CREATE COLLECTION messages (
  _id      STRING REQUIRED,
  from_id  STRING REQUIRED,
  to_id    STRING REQUIRED,
  text     STRING REQUIRED,
  sent_at  INT64   REQUIRED
) STRICT;

-- (!برگشتناپذیر) Collection حذف
DROP COLLECTION temp_sessions;

-- Schema به‌روزرسانی
ALTER COLLECTION users SET SCHEMA (
  phone STRING,
  avatar STRING
);

-- collections لیست همه
SHOW COLLECTIONS;

-- بررسی وجود
COLLECTION EXISTS users;
```

```
CREATE COLLECTION <name> [ ( <field> <type> [REQUIRED] [UNIQUE] [DEFAULT  
<val>] , ... ) ] [ STRICT ] ;
```

DROP COLLECTION تمام اسناد، ایندکس‌ها و Foreign Key های آن collection را حذف می‌کند و قابل بازگشت نیست. قبل از اجرا **BACKUP** بگیرید. ⚠

DocEngine::CreateIndex / AddForeignKey

DDL — INDEX & FK

```
-- ایندکس تکی روی یک فیلد
CREATE INDEX idx_username ON users (username);

-- ایندکس یکتا
CREATE UNIQUE INDEX idx_email ON users (email);

-- ایندکس ترکیبی (Compound)
CREATE INDEX idx_author_date ON posts (author_id, created_at);

-- حذف ایندکس
DROP INDEX idx_username ON users;

-- Foreign Key: posts.author_id → users._id
ADD FOREIGN KEY fk_post_author
  ON posts (author_id)
  REFERENCES users (_id);

-- Foreign Key: comments.post_id → posts._id
ADD FOREIGN KEY fk_comment_post
  ON comments (post_id)
  REFERENCES posts (_id);

-- حذف Foreign Key
DROP FOREIGN KEY fk_post_author ON posts;

-- نمایش ایندکس‌های یک collection
SHOW INDEXES ON users;

-- نمایش Foreign Keyها
SHOW FOREIGN KEYS ON posts;
```

Foreign Key ها نقش مضاعف دارند: هم یکپارچگی داده را در DocEngine تضمین می‌کنند، هم GraphEngine از آن‌ها برای کشف خودکار edge type در گراف استفاده می‌کند. هر FK = یک نوع edge احتمالی.



DocEngine::InsertOne / InsertMany

DML — INSERT INTO

```
-- (کامل JSON) درج یک سند --
INSERT INTO users VALUES (
  '{
    "username": "ali_reza",
    "email": "ali@example.com",
    "age": 28,
    "active": true,
    "tags": ["developer", "photographer"],
    "address": {"city": "Tehran", "country": "IR"},
    "created_at": 1700000000000
  }'
);

-- دستی _id درج با
INSERT INTO users VALUES (
  '{"_id": "u_001", "username": "sara", "email": "sara@ex.com"}'
);

-- خودکار اختصاص می‌یابد → UUID v4 بدون _id
-- درج چند سند (اتمیک - همه یا هیچ)
INSERT INTO posts BATCH VALUES
  ('{"title": "پست اول", "author_id": "u_001", "likes": 0}'),
  ('{"title": "پست دوم", "author_id": "u_001", "likes": 5}'),
  ('{"title": "پست سوم", "author_id": "u_002", "likes": 12}');

-- (مخصوص ارتباط collection) درج پال فالو --
INSERT INTO follows VALUES (
  '{"from_id": "u_001", "to_id": "u_002", "created_at": 1700001000}'
);
```

بازگشت خروجی: `INSERT INTO` شناسه سند(ها) را برمی‌گرداند. برای BATCH، آرایه‌ای از IDها. اگر Schema Validation یا FK Check شکست بخورد، خطا برمی‌گردد.




```
-- ID خواندن (سریع‌ترین -  $O(1)$ )
SELECT * FROM users WHERE _id = 'u_001';

-- جستجوی ساده
SELECT * FROM users WHERE age > 18;

-- Projection (فقط فیلدهای مشخص)
SELECT username, email, age FROM users WHERE active = true;

-- شرط ترکیبی AND
SELECT * FROM users
WHERE age >= 18
      AND active = true
      AND address.country = 'IR';

-- شرط OR
SELECT * FROM users
WHERE role = 'admin' OR role = 'moderator';

-- عملگر IN
SELECT * FROM users
WHERE status IN ('active', 'verified', 'premium');

-- بررسی وجود فیلد
SELECT * FROM users WHERE avatar EXISTS;
SELECT * FROM users WHERE avatar NOT EXISTS;

-- LIKE (regex pattern)
SELECT * FROM users WHERE username LIKE '^ali';
SELECT * FROM users WHERE email CONTAINS 'gmail';
SELECT * FROM users WHERE username STARTS WITH 'ali';

-- دسترسی به فیلدهای تودرتو
SELECT * FROM users WHERE address.city = 'Tehran';

-- Pagination (limit + skip)
SELECT * FROM posts
WHERE likes > 0
LIMIT 20 SKIP 40; -- صفحه سوم

-- شمارش
COUNT FROM users WHERE active = true;
COUNT FROM users; -- از شمارنده داخلی  $O(1)$ 
```

-- COUNT بهینه‌تر از بررسی وجود حداقل یک نتیجه

```
EXISTS IN users WHERE email = 'ali@ex.com';
```

```
SELECT * | <field1>, <field2>, ... FROM <collection> [ WHERE <condition>  
[AND|OR <condition>] ... ] [ LIMIT <n> ] [ SKIP <n> ] ;
```

DocEngine::UpdateById / UpdateMany

DML — UPDATE

```

-- به‌روزرسانی با ID
UPDATE users SET bio = 'سلام دنیا' WHERE _id = 'u_001';

-- چند فیلد هم‌زمان
UPDATE users
SET bio = 'به‌روز شد',
    active = true,
    updated_at = NOW()
WHERE _id = 'u_001';

-- افزایش عددی ($inc)
UPDATE posts INCREMENT likes BY 1 WHERE _id = 'p_abc';
UPDATE posts INCREMENT views BY 1, likes BY -1 WHERE _id = 'p_abc';

-- حذف فیلد ($unset)
UPDATE users UNSET temp_token WHERE _id = 'u_001';

-- افزودن به آرایه ($push)
UPDATE users PUSH tags = 'photography' WHERE _id = 'u_001';

-- افزودن یکتا ($addToSet)
UPDATE posts ADD TO SET liked_by = 'u_002' WHERE _id = 'p_001';

-- حذف از آرایه ($pull)
UPDATE users PULL tags = 'old_tag' WHERE _id = 'u_001';

-- ضرب ($mul)
UPDATE products MULTIPLY price BY 1.1 WHERE category = 'food';

-- حداقل/حداکثر ($min/$max)
UPDATE scores SET MIN value = 0 WHERE _id = 's_001';

-- به‌روزرسانی چند سند با شرط
UPDATE MANY users SET active = false
WHERE last_login < 1680000000000;

```

```

UPDATE <collection> SET <field> = <value> [, ...] WHERE <condition> ;
UPDATE <collection> INCREMENT <field> BY <n> WHERE <condition> ; UPDATE

```

```
<collection> PUSH <array_field> = <value> WHERE <condition> ; UPDATE MANY  
<collection> ... ;
```

DocEngine::DeleteById / DeleteMany

DML — DELETE

```
-- حذف با ID  
DELETE FROM users WHERE _id = 'u_bad';  
  
-- حذف چند سند با شرط  
DELETE FROM sessions WHERE expires_at < 1700000000;  
  
-- حذف همه (احتیاط!)  
DELETE FROM temp_logs WHERE true;
```

DocEngine::LookupJoin

DQL — LOOKUP JOIN

-- پستها را با اطلاعات نویسنده‌شان ترکیب کن

```
SELECT * FROM posts
LOOKUP JOIN users
    ON posts.author_id = users._id
WHERE posts.likes > 100
LIMIT 20;
```

-- کامنت‌ها را با پست و نویسنده ترکیب کن

```
SELECT * FROM comments
LOOKUP JOIN posts  ON comments.post_id = posts._id
LOOKUP JOIN users  ON comments.user_id = users._id
WHERE comments.created_at > 1700000000
LIMIT 50;
```

-- مرجع collection با داده‌های __joined__ نتیجه: هر سند شامل فیلد

```
SELECT * FROM <from_col> LOOKUP JOIN <to_col> ON <from_col.field> =
<to_col.field> [ WHERE <condition> ] [ LIMIT <n> ] ;
```


DocEngine::BeginTransaction / Commit / Rollback

TCL — TRANSACTION

```
-- الگوی پایه
BEGIN TRANSACTION;

INSERT INTO messages VALUES (
    '{"from_id":"u_001","to_id":"u_002","text":"سلام"}'
);

UPDATE users INCREMENT message_count BY 1
    WHERE _id = 'u_001';

COMMIT;

-- در صورت خطا Rollback با
BEGIN TRANSACTION;

INSERT INTO orders VALUES ('{"user_id":"u_001","total":150000}');
UPDATE wallets INCREMENT balance BY -150000
    WHERE user_id = 'u_001';
COMMIT;

-- اجرا می‌شود ROLLBACK: در صورت خطا
ROLLBACK;

-- دیده می‌شود (uncommitted changes) خواندن درون تراکنش
BEGIN TRANSACTION;

INSERT INTO posts VALUES ('{"title":"draft"}');

SELECT * FROM posts WHERE title = 'draft' IN TRANSACTION;

ROLLBACK;
```

دو نوع گراف: **LIVE** همواره در RAM است و تغییرات لحظه‌ای دارد. **STATIC** روی دیسک است و فقط هنگام اجرای job سنگین به RAM منتقل می‌شود و بعد آزاد می‌شود.

LiveGraph — GraphStorage — GraphWAL

GDL — CREATE GRAPH

```
-- (تغییر لحظه‌ای، RAM همیشه در) گراف پویا
CREATE LIVE GRAPH social_graph;

-- (RAM در job روی دیسک، فقط هنگام) گراف ثابت
CREATE STATIC GRAPH analytics_graph;

-- (فقط زیرمجموعه‌ای از داده‌ها) WHERE گراف با شرط
CREATE LIVE GRAPH active_users_graph
WHERE users.active = true;

-- (edge و node چند نوع - heterogeneous) گراف ناممکن
CREATE LIVE GRAPH full_social
    HETEROGENEOUS -- پشتیبانی از انواع مختلف node/edge
    DIRECTED;      -- یال‌ها جهت‌دار هستند (پیش‌فرض)

-- گراف بدون جهت
CREATE LIVE GRAPH similarity_graph
    UNDIRECTED;

-- (current graph تنظیم) استفاده از گراف
USE GRAPH social_graph;

-- نمایش همه گراف‌ها
SHOW GRAPHS;

-- اطلاعات یک گراف
DESCRIBE GRAPH social_graph;

-- (حذف می‌شوند nex/.nexr/.nexl، فایل‌های) حذف گراف
DROP GRAPH analytics_graph;
```

این بخش مهم‌ترین بخش NexoraQL است. کاربر مشخص می‌کند که کدام فیلدهای collection ها به node و کدام به edge تبدیل شوند. سه روش اصلی وجود دارد.

**LiveGraph::addNode + TypeRegistry**

تعریف گره — GDL — MAP NODE

```
-- یک گره = collection الگو: هر سند در
USE GRAPH social_graph;

-- node نوع User از collection users
MAP NODE User
  FROM users
  KEY _id          -- از آن ساخته می‌شود DenseId فیلدی که
  PROPERTIES username, avatar, age; -- فیلدهایی که در گراف کپی می‌شوند

-- node نوع Post از collection posts
MAP NODE Post
  FROM posts
  KEY _id
  PROPERTIES title, created_at, likes;

-- node (فقط کاربران فعال)
MAP NODE ActiveUser
  FROM users
  KEY _id
  PROPERTIES username
  WHERE active = true;

-- node با embedding (برای ML/GNN)
MAP NODE Post
  FROM posts
  KEY _id
  PROPERTIES title, embedding; -- embedding: ARRAY of float
```

مجزا برای رابطه Collection: روش ۱

مثال: collection به نام follows با فیلد from_id و to_id

```
-- سند follows: {"from_id":"u_001","to_id":"u_002","created_at":...}
```

```
MAP EDGE FOLLOWS
```

```
FROM follows -- collection مبدأ
```

```
SOURCE from_id AS User -- فیلد مبدأ → node type
```

```
TARGET to_id AS User -- فیلد مقصد → node type
```

```
DIRECTED
```

```
PROPERTIES created_at;
```

```
-- سند likes: {"user_id":"u_001","post_id":"p_xyz","timestamp":...}
```

```
MAP EDGE LIKED
```

```
FROM likes
```

```
SOURCE user_id AS User
```

```
TARGET post_id AS Post
```

```
DIRECTED;
```

روش ۲: فیلد داخل سند به سند دیگر اشاره میکند
مثال: `posts.author_id → users._id`

```
-- سند post: {"_id":"p_001","author_id":"u_001","title":"..."}

MAP EDGE AUTHORED

  FROM posts

  SOURCE author_id AS User      -- author_id در posts به User اشاره دارد
  TARGET _id        AS Post     -- post هدف: خود همین سند

  DIRECTED

  PROPERTIES created_at;

-- سند comment: {"_id":"c_001","post_id":"p_001","user_id":"u_002",...}

MAP EDGE COMMENTED_ON

  FROM comments

  SOURCE user_id AS User
  TARGET post_id AS Post

  DIRECTED;

-- فیلد تودرتو
-- سند: {"_id":"p_001","meta":{"creator_id":"u_001"}}

MAP EDGE CREATED_BY

  FROM posts

  SOURCE meta.creator_id AS User
  TARGET _id              AS Post

  DIRECTED;
```

روش ۲: یک آرایه در سند = چند یال
چند edge (UNWIND) → document یک

```
-- سند user: {"_id":"u_001","following":["u_002","u_003","u_004"]}
-- یعنی u_001 فالو کرده u_002، u_003، u_004
MAP EDGE FOLLOWS
  FROM users
  UNWIND following AS followed_id -- را باز کن following آرایه
  SOURCE _id AS User -- مبدأ: خود سند
  TARGET followed_id AS User -- مقصد: هر آیتم آرایه
  DIRECTED;
-- 3 نتیجه: u_001→u_002, u_001→u_003, u_001→u_004
```

```
-- سند post: {"_id":"p_001","liked_by":["u_001","u_003"]}
MAP EDGE LIKED
  FROM posts
  UNWIND liked_by AS liker_id
  SOURCE liker_id AS User
  TARGET _id AS Post
  DIRECTED;
-- نتیجه: u_001→p_001, u_003→p_001
```

```
-- UNWIND فیلد تودرتو
-- سند: {"_id":"p_001","engagement":{"likedBy":["u_001","u_002"]}}
MAP EDGE LIKED
  FROM posts
  UNWIND engagement.likedBy AS liker_id
  SOURCE liker_id AS User
  TARGET _id AS Post
  DIRECTED;
```

```
-- (object array UNWIND) آرایه ای از آبجکت
-- سند: {"_id":"u_001","connections":[{"id":"u_002","since":1700}]}
MAP EDGE CONNECTED
  FROM users
  UNWIND connections AS conn
  SOURCE _id AS User
  TARGET conn.id AS User -- دسترسی به فیلد داخل آبجکت آرایه
  DIRECTED
  PROPERTIES conn.since;
```

```
-- ----- تعریف گراف -----
CREATE LIVE GRAPH social_network HETEROGENEOUS DIRECTED;
USE GRAPH social_network;

-- ----- تعریف node types -----
MAP NODE User FROM users KEY _id PROPERTIES username, avatar;
MAP NODE Post FROM posts KEY _id PROPERTIES title, created_at;
MAP NODE Comment FROM comments KEY _id PROPERTIES text;

-- ----- مجزا collection: روش ۱ -----
MAP EDGE Follows
  FROM follows_table
  SOURCE from_user_id AS User
  TARGET to_user_id AS User
  DIRECTED PROPERTIES since;

-- ----- در سند FK: روش ۲ -----
MAP EDGE Authored
  FROM posts
  SOURCE author_id AS User
  TARGET _id AS Post
  DIRECTED PROPERTIES created_at;

-- ----- آرایه UNWIND: روش ۳ -----
MAP EDGE Liked_Post
  FROM posts
  UNWIND liked_by AS liker
  SOURCE liker AS User
  TARGET _id AS Post
  DIRECTED;

MAP EDGE Wrote_Comment
  FROM comments
  SOURCE user_id AS User
  TARGET post_id AS Post
  DIRECTED;
```



```
-- ===== BUILD: ساخت فایل‌های پایتری از RocksDB =====
-- را روی دیسک می‌سازد nexr و nex.
-- اجرا شود RENDER باید قبل از
BUILD GRAPH social_network;

-- Build با verbose output
BUILD GRAPH social_network VERBOSE;

-- Build فقط یک node type
BUILD GRAPH social_network NODES ONLY;

-- Build فقط edges
BUILD GRAPH social_network EDGES ONLY;

-- ===== RENDER: بارگذاری گراف در RAM =====
-- را در حافظه می‌سازد LiveGraph را می‌خواند و nexr/nex. فایل‌های
-- می‌شوند render خودکار startup ها در LIVE graph
RENDER GRAPH social_network;

-- ===== REFRESH: rebuild داده‌های جدید =====
-- ها یا زمانی که داده‌ها تغییر زیادی کرده‌اند STATIC graph برای
REFRESH GRAPH social_network;

-- REFRESH (برای job scheduler) با بازه زمانی
REFRESH GRAPH analytics EVERY 6 HOURS;

-- ===== STATUS =====
GRAPH STATUS social_network;
-- خروجی: {state: "Clean", nodes: 50000, edges: 1200000, version: 42}

-- ===== Compaction (در زمان خلوت) =====
-- را از دیسک حذف می‌کند DELETED رکوردهای
COMPACT GRAPH social_network;

-- ===== WAL management =====
PURGE GRAPH WAL social_network;           -- قدیمی را پاک کن WAL
GRAPH WAL STATUS social_network;          -- وضعیت WAL
REPLAY GRAPH WAL social_network;          -- دستی (recovery) replay

-- ===== Snapshot برای STATIC graph =====
-- (برای job) می‌سازد LiveGraph یک کپی فشرده از
SNAPSHOT GRAPH social_network INTO analytics;
```

جریان کامل:  → BUILD GRAPH → MAP NODE / MAP EDGE → CREATE LIVE GRAPH → RENDER GRAPH → گراف آماده استفاده. برای LIVE graph ها، RENDER در startup خودکار است.



```
-- همسایه های مستقیم (depth=1)
TRAVERSE User("u_001")
  OUT FOLLOWS
  DEPTH 1
  LIMIT 50;

-- فالورهای یک کاربر (incoming)
TRAVERSE User("u_001")
  IN FOLLOWS
  DEPTH 1
  LIMIT 100;

-- هر دو جهت
TRAVERSE User("u_001")
  BOTH FOLLOWS
  DEPTH 1;

-- traversal عمیقتر (depth=2: دوستان دوستان)
TRAVERSE User("u_001")
  OUT FOLLOWS
  DEPTH 2
  LIMIT 200;

-- type بدون فیلتر همه یالها
TRAVERSE User("u_001")
  OUT *
  DEPTH 1
  LIMIT 100;

-- پستهای یک کاربر (User → Post)
TRAVERSE User("u_001")
  OUT AUTHORED
  DEPTH 1
  LIMIT 20;

-- node اطلاعات یک
GET NODE User("u_001");

-- بررسی وجود یال
EDGE EXISTS User("u_001") -[FOLLOWS]-> User("u_002");

-- آمار گراف
GRAPH STATS social_network;
```

```
TRAVERSE <NodeType>(<id>) OUT | IN | BOTH <EdgeType> | * DEPTH <n> [ LIMIT  
<n> ] ;
```

دو حالت اجرا: `RUN LOCK` برای الگوریتم‌های سبک روی LiveGraph (blocking, سریع). `RUN JOB` برای الگوریتم‌های سنگین روی snapshot در background thread (async, نتیجه با `JOB STATUS`).

LockAlgorithm — LiveGraph shared_lock

GAL — RUN LOCK (سبک، روی LIVEGRAPH)

```
-- فالورهای مشترک دو کاربر
RUN LOCK CommonFollowers ON social_network
  WITH user1 = 'u_001', user2 = 'u_002'
  LIMIT 100;

-- آیا دو کاربر ارتباط دارند؟
RUN LOCK AreConnected ON social_network
  WITH user1 = 'u_001', user2 = 'u_005';

-- پیشنهاد افراد برای فالو (دوستان دوستان)
RUN LOCK SuggestFollowers ON social_network
  WITH user = 'u_001', depth = 2
  LIMIT 20;

-- node کوتاه‌ترین مسیر بین دو
RUN LOCK ShortestPath ON social_network
  WITH from = 'u_001', to = 'u_999',
       edge_type = 'FOLLOWS',
       max_depth = 6;

-- تعداد فالورهای مشترک بین چند کاربر
RUN LOCK CommonFollowers ON social_network
  WITH users = ['u_001', 'u_002', 'u_003'];
```

```
-- PageRank روی کل گراف (async)
RUN JOB PageRank ON social_network
  WITH iterations = 20, damping = 0.85
  RETURNS top 100;

-- Community Detection
RUN JOB CommunityDetection ON social_network
  WITH algorithm = 'louvain',
       min_community_size = 10;

-- تحلیل شبکه کامل
RUN JOB NetworkAnalysis ON social_network
  WITH metrics = ['degree', 'betweenness', 'clustering'];

-- بررسی وضعیت job
JOB STATUS 'job_abc123';

-- خروجی: {id:"job_abc123", status:"running", progress:65, eta:120}

-- گرفتن نتیجه job
JOB RESULT 'job_abc123';

-- لغو job
JOB CANCEL 'job_abc123';

-- job لیست همه
SHOW JOBS;

SHOW JOBS ON social_network;
```

● جدول الگوریتم‌ها / Algorithm Reference

نام	نوع	پارامترها	کاربرد
CommonFollowers	LOCK	user1, user2, [limit]	فالورهای مشترک دو یا چند کاربر
SuggestFollowers	LOCK	user, depth, limit	پیشنهاد برای فالو کردن
ShortestPath	LOCK	from, to, edge_type, max_depth	کوتاه‌ترین مسیر بین دو node
AreConnected	LOCK	user1, user2	آیا ارتباط مستقیم وجود دارد؟
Neighborhood	LOCK	node, depth, limit	زیرگراف محلی یک node
PageRank	JOB	iterations, damping, top_n	رتبه‌بندی تأثیرگذاری کاربران
CommunityDetection	JOB	algorithm, min_size	کشف گروه‌های جامعه‌شناختی
NetworkAnalysis	JOB	[]metrics	آمار کامل شبکه
Centrality	JOB	type (degree/betweenness/closeness)	مرکزیت nodes

DocEngine + GraphStorage + WAL

SYS — SYSTEM ADMIN

```
-- وضعیت سیستم
SYSTEM STATUS;

-- خروجی: {healthy: true, rocksdb: "ok", graphs: 3, uptime: "2h 30m"}

-- مدیریت Collections
SHOW COLLECTIONS;
COLLECTION EXISTS users;
DESCRIBE COLLECTION users; -- Schema + Index + FK

-- مدیریت گرافها
SHOW GRAPHS;
DESCRIBE GRAPH social_network;
GRAPH STATUS social_network;

-- WAL
GRAPH WAL STATUS social_network;
PURGE GRAPH WAL social_network;
REPLAY GRAPH WAL social_network; -- recovery دستی

-- فشرده سازی
COMPACT GRAPH social_network; -- از دیسک DELETED حذف رکوردهای
COMPACT ALL GRAPHS;

-- اطلاعات سیستم
SYSTEM INFO;

-- خروجی: {version:"1.0.0", rocksdb_version:"...", graphs_in_ram:[...], ...}
```

Quick Reference Cheatsheet / جدول سریع

Document Store

CREATE COLLECTION

DROP COLLECTION

INSERT INTO ... VALUES

SELECT * FROM ... WHERE

UPDATE ... SET ... WHERE

DELETE FROM ... WHERE

... COUNT FROM

... EXISTS IN

CREATE INDEX

ADD FOREIGN KEY

LOOKUP JOIN

BEGIN / COMMIT / ROLLBACK

Graph Definition

CREATE LIVE GRAPH

CREATE STATIC GRAPH

USE GRAPH

MAP NODE ... FROM

MAP EDGE ... FROM ... SOURCE ... TARGET

MAP EDGE ... UNWIND

BUILD GRAPH

RENDER GRAPH

REFRESH GRAPH

	COMPACT GRAPH
	DROP GRAPH
Graph Query & Algo 🔍	
	TRAVERSE ... OUT/IN/BOTH
	... DEPTH ... LIMIT
	GET NODE
	EDGE EXISTS
	GRAPH STATS
	<RUN LOCK <Algo
	<RUN JOB <Algo
	JOB STATUS
	JOB RESULT
	JOB CANCEL
	SHOW JOBS